Lecture and hands-on tutorial

Graph Neural Networks

Shah Rukh Qasim

Department of Mathematical Modeling and Machine Learning
Physik Institut, University of Zurich

shahrukh.qasim@physik.uzh.ch

27 October 2026

Today's agenda

Part 1: Lecture

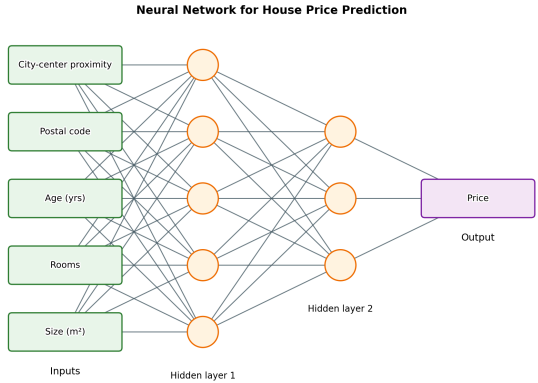
- Why graph neural networks?
- Modern GNNs and message passing
- Understanding the components needed to use a GNN
- A short history of GNNs
- Advanced topics: dynamic graphs, scaling and heterogeneous graphs

Part 2: Tutorial

- Train a GNN for molecular classification in Colab
- Implement a message-passing layer
- Build a graph classifier
- Adapt the notebook to future graph problems

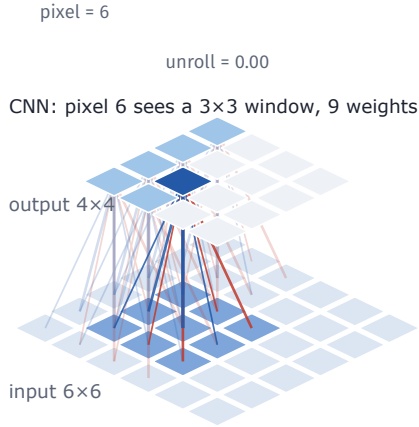
Fully connected neural networks

- Start with a simple fully connected neural network.
- Each input is connected to the next layer.
- A simple, widely used architecture.



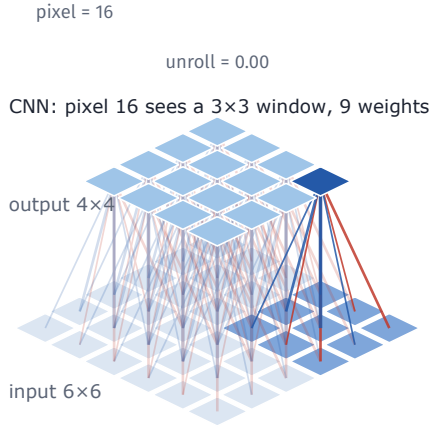
Images have structure

- Consider image classification: cat or dog?
- A 1000 × 1000 image has 10^6 input values per channel.
- Connecting 10^6 inputs to 10^6 hidden units requires 10^{12} weights.
- Convolutional neural networks exploit the local structure of images.
- CNN: a 3 × 3 window; the same 9 weights slide over the image.
- MLP: flatten the image, connect everything.



Images have structure

- Consider image classification: cat or dog?
- A 1000 × 1000 image has 10^6 input values per channel.
- Connecting 10^6 inputs to 10^6 hidden units requires 10^{12} weights.
- Convolutional neural networks exploit the local structure of images.
- CNN: a 3 × 3 window; the same 9 weights slide over the image.
- MLP: flatten the image, connect everything.



Images have structure

- Consider image classification: cat or dog?
- A 1000 × 1000 image has 10^6 input values per channel.
- Connecting 10^6 inputs to 10^6 hidden units requires 10^{12} weights.
- Convolutional neural networks exploit the local structure of images.
- CNN: a 3 × 3 window; the same 9 weights slide over the image.
- MLP: flatten the image, connect everything.



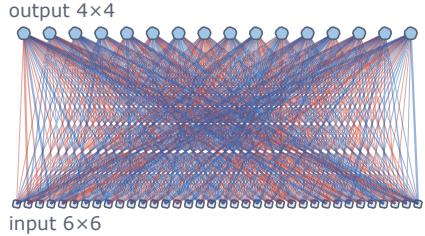
Images have structure

- Consider image classification: cat or dog?
- A 1000 × 1000 image has 10^6 input values per channel.
- Connecting 10^6 inputs to 10^6 hidden units requires 10^{12} weights.
- Convolutional neural networks exploit the local structure of images.
- CNN: a 3 × 3 window; the same 9 weights slide over the image.
- MLP: flatten the image, connect everything.

pixel = 16

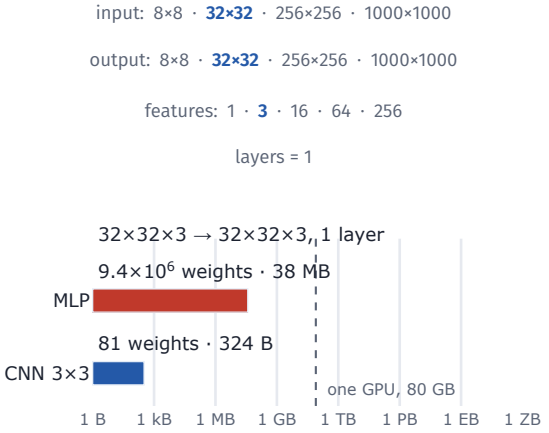
unroll = 1.00

MLP: every input to every output, 576 weights



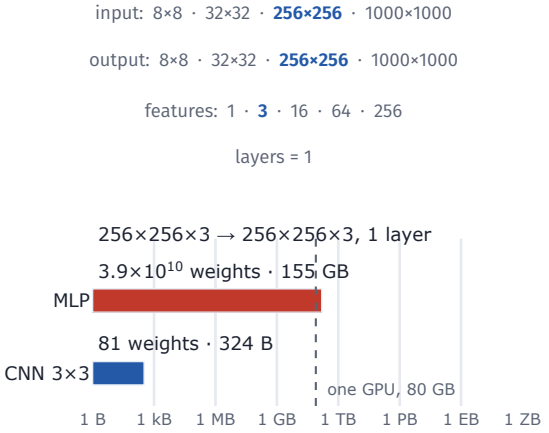
Images have structure

- Consider image classification: cat or dog?
- A 1000 × 1000 image has 10^6 input values per channel.
- Connecting 10^6 inputs to 10^6 hidden units requires 10^{12} weights.
- Convolutional neural networks exploit the local structure of images.
- CNN: a 3 × 3 window; the same 9 weights slide over the image.
- MLP: flatten the image, connect everything.



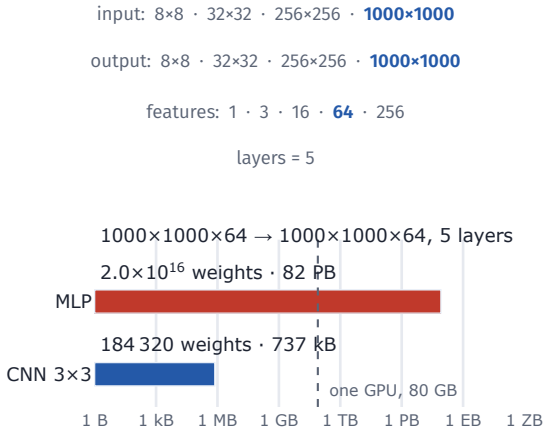
Images have structure

- Consider image classification: cat or dog?
- A 1000 × 1000 image has 10^6 input values per channel.
- Connecting 10^6 inputs to 10^6 hidden units requires 10^{12} weights.
- Convolutional neural networks exploit the local structure of images.
- CNN: a 3 × 3 window; the same 9 weights slide over the image.
- MLP: flatten the image, connect everything.



Images have structure

- Consider image classification: cat or dog?
- A 1000 × 1000 image has 10^6 input values per channel.
- Connecting 10^6 inputs to 10^6 hidden units requires 10^{12} weights.
- Convolutional neural networks exploit the local structure of images.
- CNN: a 3 × 3 window; the same 9 weights slide over the image.
- MLP: flatten the image, connect everything.



Why graph neural networks?

- Images: a regular pixel grid.
- Tables: one feature vector.

A lot of data is hard to represent as either.

Example: Zurich trams

- **Nodes**: tram stops.
- **Edges**: a tram runs directly between two stops.



Zurich tram network: © OpenStreetMap contributors (ODbL); map © swisstopo.

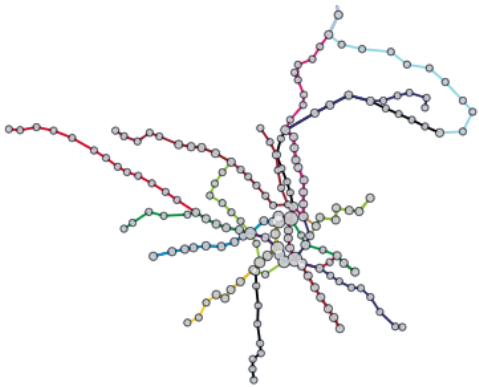
Why graph neural networks?

- Images: a regular pixel grid.
- Tables: one feature vector.

A lot of data is hard to represent as either.

Example: Zurich trams

- **Nodes**: tram stops.
- **Edges**: a tram runs directly between two stops.



Zurich tram network: © OpenStreetMap contributors (ODbL); map © swisstopo.

Why graph neural networks?

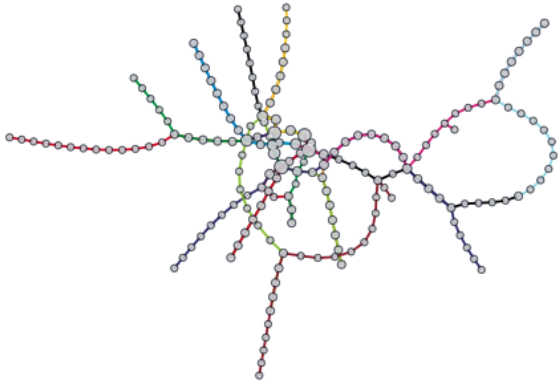
- Images: a regular pixel grid.
- Tables: one feature vector.

A lot of data is hard to represent as either.

Example: Zurich trams

- **Nodes**: tram stops.
- **Edges**: a tram runs directly between two stops.

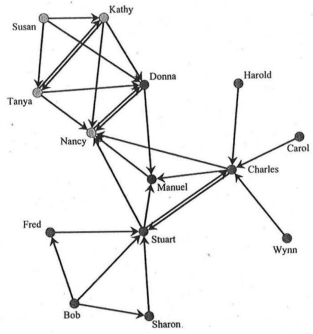
view: Map · Graph only · **Rearranged**



Zurich tram network: © OpenStreetMap contributors (ODbL); map © swisstopo.

Social networks

- **Nodes**: people.
- **Edges**: friendships.
- Each person has a different number of friends.
- People have no natural order.
- Task: predict possible friendships.



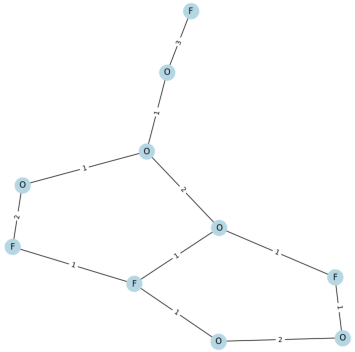
Social-network illustration: J. Clyde Mitchell, “Social networks,” Annual Review of Anthropology 3 (1974), 279–299.

Molecules

- **Nodes**: atoms.
- **Edges**: chemical bonds.
- Molecules differ in size and shape: no fixed-length vector, no pixel grid.
- Task: predict molecular properties, such as solubility or toxicity.
- We will classify molecules in the tutorial.

Relationships, but no pixel grid.

Render them as images for a CNN, or use their structure directly?

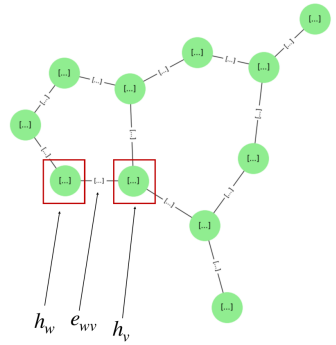


Molecular graph from the source lecture.

Graphs and features

- A graph is $G = (V, E)$.
- **Nodes** V represent objects; **edges** E represent relationships.
- A node has a feature vector h_v; for a person, this might encode age and other attributes.
- An edge may have features e_{wv}.
- Neural networks operating on this representation are GNNs.
- Sets, images and sequences can all be represented as graphs.

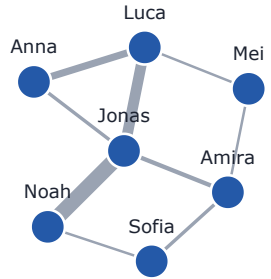
example: **Notation** · Friends · Molecule



Graphs and features

- A graph is $G = (V, E)$.
- **Nodes** V represent objects; **edges** E represent relationships.
- A node has a feature vector h_v; for a person, this might encode age and other attributes.
- An edge may have features e_{wv}.
- Neural networks operating on this representation are GNNs.
- Sets, images and sequences can all be represented as graphs.

example: Notation · **Friends** · Molecule



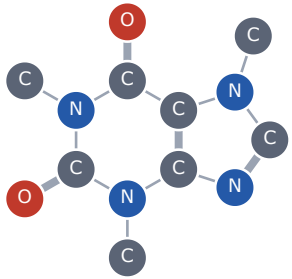
h_v = [age, years in Zurich, student]

e_{wv} = [years of friendship, messages per week]

Graphs and features

- A graph is $G = (V, E)$.
- **Nodes** V represent objects; **edges** E represent relationships.
- A node has a feature vector h_v; for a person, this might encode age and other attributes.
- An edge may have features e_{wv}.
- Neural networks operating on this representation are GNNs.
- Sets, images and sequences can all be represented as graphs.

example: Notation · Friends · **Molecule**



h_v = [C, N, O, attached H, in ring]

e_{wv} = [single, double, in ring]

Graph message passing

- Message passing is the core idea behind modern GNNs.
- Each node sends messages derived from its features or hidden representation.
- A node aggregates messages from its neighbours and updates its own representation.

Notation

We follow the message-passing neural network framework of Gilmer et al.

Justin Gilmer et al., “Neural message passing for quantum chemistry,” International Conference on Machine Learning, PMLR (2017).

Messages, aggregation and updates

Message from node w to node v:

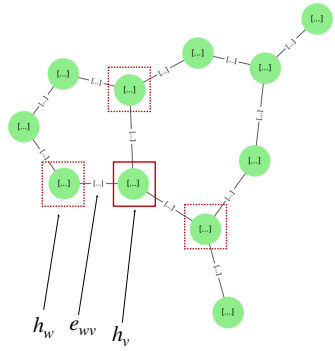
$$m_{wv}^t = M_t(h_w^t, h_v^t, e_{wv}).$$

Aggregate over the neighbourhood:

$$m_v^{t+1} = \sum_{w \in N(v)} M_t(h_w^t, h_v^t, e_{wv}).$$

Sum, mean, min or max: the aggregation should be permutation invariant.

$$h_v^{t+1} = U_t(h_v^t, m_v^{t+1}).$$

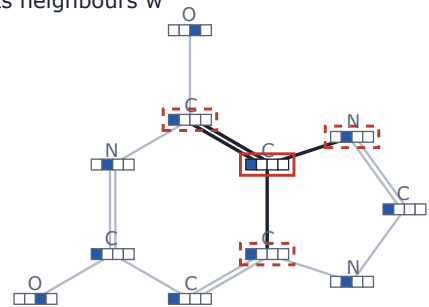


Message passing, step by step

One node *v*: its neighbours send messages, *v* aggregates them and updates its state.

node v = 3 stage = 0.00 **sum** · mean · max

node v and its neighbours w



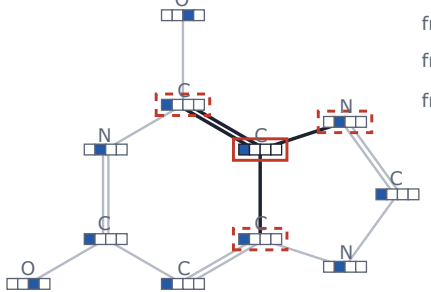
Molecule from the tutorial's dataset: AIDS Antiviral Screen Data, IAM Graph Database (Riesen and Bunke, 2008).

Message passing, step by step

One node *v*: its neighbours send messages, *v* aggregates them and updates its state.

node v = 3 stage = 1.00 **sum** · mean · max

1 messages $m_{wv} = M(h_w, h_v, e_{wv})$





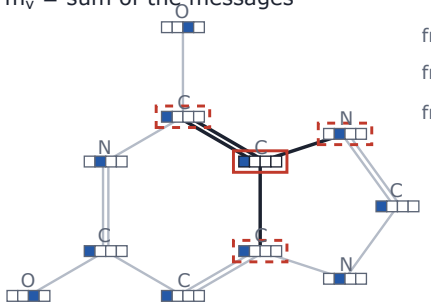
Molecule from the tutorial's dataset: AIDS Antiviral Screen Data, IAM Graph Database (Riesen and Bunke, 2008).

Message passing, step by step

One node *v*: its neighbours send messages, *v* aggregates them and updates its state.

node v = 3 stage = 2.00 **sum** · mean · max

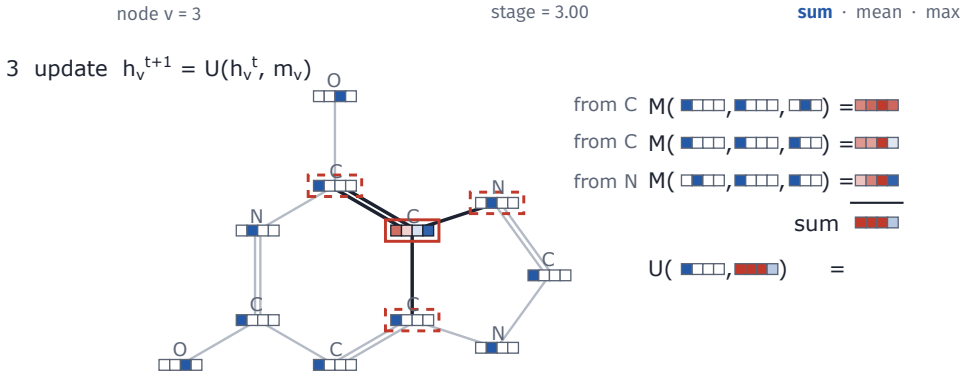
2 aggregate m_v = sum of the messages



Molecule from the tutorial's dataset: AIDS Antiviral Screen Data, IAM Graph Database (Riesen and Bunke, 2008).

Message passing, step by step

One node *v*: its neighbours send messages, *v* aggregates them and updates its state.



Molecule from the tutorial's dataset: AIDS Antiviral Screen Data, IAM Graph Database (Riesen and Bunke, 2008).

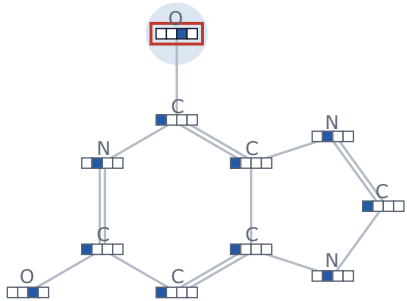
Message passing, step by step

Every round, all nodes update at once: after *t* rounds, *v* has seen everything *t* bonds away.

node v = 1

round = 0.00

round 0: one-hot atom types



Molecule from the tutorial's dataset: AIDS Antiviral Screen Data, IAM Graph Database (Riesen and Bunke, 2008).

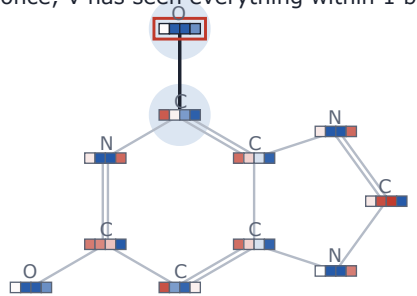
Message passing, step by step

Every round, all nodes update at once: after *t* rounds, *v* has seen everything *t* bonds away.

node v = 1

round = 1.00

round 1: all atoms update at once; v has seen everything within 1 bond



Molecule from the tutorial's dataset: AIDS Antiviral Screen Data, IAM Graph Database (Riesen and Bunke, 2008).

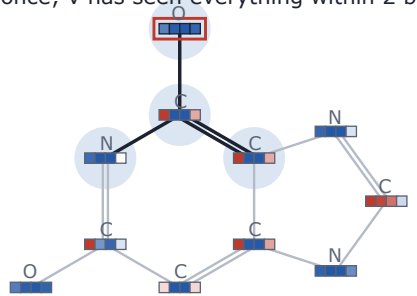
Message passing, step by step

Every round, all nodes update at once: after *t* rounds, *v* has seen everything *t* bonds away.

node v = 1

round = 2.00

round 2: all atoms update at once; v has seen everything within 2 bonds



Molecule from the tutorial's dataset: AIDS Antiviral Screen Data, IAM Graph Database (Riesen and Bunke, 2008).

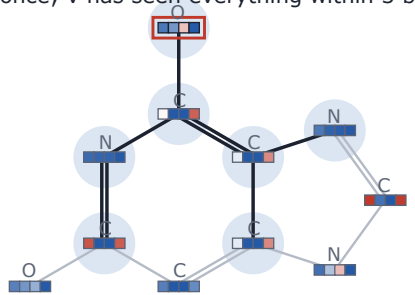
Message passing, step by step

Every round, all nodes update at once: after *t* rounds, *v* has seen everything *t* bonds away.

node v = 1

round = 3.00

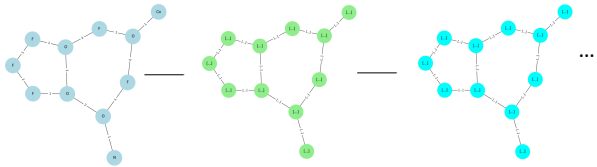
round 3: all atoms update at once; v has seen everything within 3 bonds



Molecule from the tutorial's dataset: AIDS Antiviral Screen Data, IAM Graph Database (Riesen and Bunke, 2008).

Stacking graph neural network layers

- Message passing produces a new set of node features.
- Repeat across several layers.
- Each layer can learn different message and update functions.
- As in a CNN, features become progressively transformed.
- The graph topology usually stays fixed.



Training for graph-level tasks

- The prediction head depends on the task.
- For molecular classification, predict one label for the entire graph.
- Pool information from all nodes: mean, min or max.

$$h_G = \mathrm{POOL}_{v \in V} h_v^T, \qquad \hat{y}_G = f_{\mathrm{head}}(h_G).$$

- Use the usual training machinery: cross-entropy loss and an optimiser such as Adam.

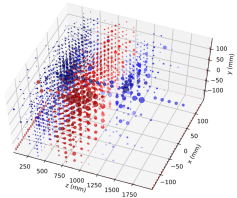
Node-level and edge-level tasks

Node-level predictions

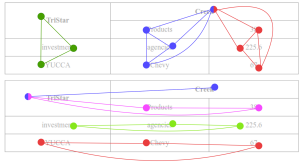
- Use each node's final feature vector.
- Example: segmentation of detector hits.

Edge-level predictions

- Predict at each edge rather than pooling the graph.
- Use edge features, or concatenate or subtract the endpoint features.
- Example: predict whether two people are friends.



Two particles



S. R. Qasim et al., "Learning representations of irregular particle-detector geometry with distance-weighted graph networks," EPJC 79 (2019). S. R. Qasim, H. Mahmood and F. Shafait, "Rethinking table recognition using graph neural networks," ICDAR (2019).

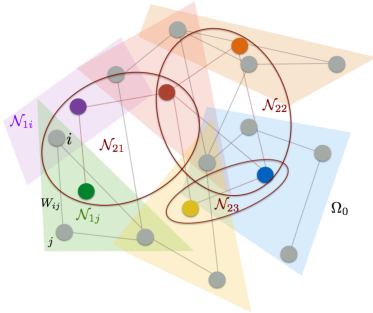
GNNs in practice: storage and batching

Quantity	Tensor	Shape
Node features	`x`	$[N, F]$
Directed edge indices	`edge_index`	$[2, E]$
Optional edge features	`edge_attr`	$[E, D]$
Graph membership	`batch`	$[N]$

- Each edge stores zero-based source and destination node indices.
- Concatenate node features: $x_{\text{batch}} = \text{cat}(x^{(1)}, x^{(2)}, \ldots)$.
- Shift each graph's edge indices by the cumulative number of preceding nodes, then concatenate.
- The batch vector records which graph each node belongs to.
- PyG provides utilities for bundling and unbundling graphs.

Spectral networks

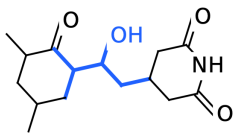
- Bruna, LeCun and collaborators sought to generalise CNNs beyond regular arrays.
- Replace the regular-grid construction with operations based on the graph spectrum.
- Use hierarchical clustering to build representations at different scales.



Joan Bruna et al., “Spectral networks and locally connected networks on graphs,” arXiv:1312.6203 (2013).

Graph convolutions and message passing

- Duvenaud et al. introduced a convolutional network operating directly on molecular graphs.
- Neighbourhood operations build a learned molecular fingerprint.
- Different terminology, with ideas closely related to message passing.



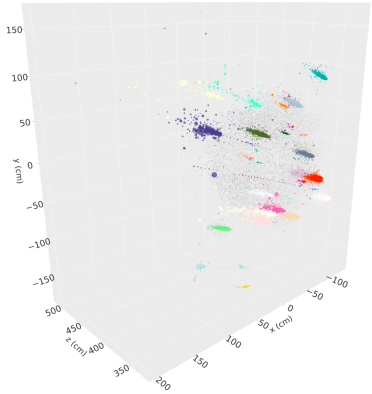
Algorithm 2 Neural graph fingerprints

Input: molecule, radius R, hidden weights $H_1^1 \dots H_R^5$, output weights $W_1 \dots W_R$
Initialize: fingerprint vector $\mathbf{f} \leftarrow \mathbf{0}_S$
for each atom a in molecule
  $\mathbf{r}_a \leftarrow g(a)$ ▷ lookup atom features
for $L = 1$ to R ▷ for each layer
  for each atom a in molecule
    $\mathbf{r}_1 \dots \mathbf{r}_N = \text{neighbors}(a)$
    $\mathbf{v} \leftarrow \mathbf{r}_a + \sum_{i=1}^{N} \mathbf{r}_i$ ▷ sum
    $\mathbf{r}_a \leftarrow \sigma(\mathbf{v} H_L^N)$ ▷ smooth function
    $\mathbf{i} \leftarrow \text{softmax}(\mathbf{r}_a W_L)$ ▷ sparsify
    $\mathbf{f} \leftarrow \mathbf{f} + \mathbf{i}$ ▷ add to fingerprint
Return: real-valued vector $\mathbf{f}$

David K. Duvenaud et al., "Convolutional networks on graphs for learning molecular fingerprints," Advances in Neural Information Processing Systems 28 (2015).

Point clouds

- Sometimes the data are an unordered set of points.
- No graph connectivity is supplied.
- Examples in physics:
  - calorimeter hits;
  - tracker measurements;
  - particles in a jet.
- How can we apply machine learning to these data?



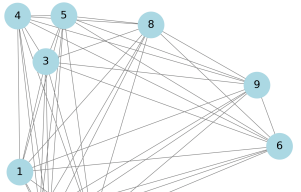
Building a graph from points

Connect every pair

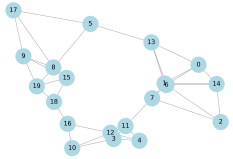
- A fully connected graph has $O(N^2)$ edges.
- This becomes expensive as the number of points grows.

Connect locally

- Use a radius in the input space.
- Or connect the k nearest neighbours (kNN).



Fully connected graph



kNN graph

Dynamic graph neural networks

1. Begin with a feature vector for each point.
2. Transform the features into a learned latent space h_v.
3. Build a kNN graph using Euclidean distances in that space.
4. Perform message passing with permutation-invariant aggregation.
5. Repeat: the neighbourhoods can change as representations change.

A learned neighbourhood

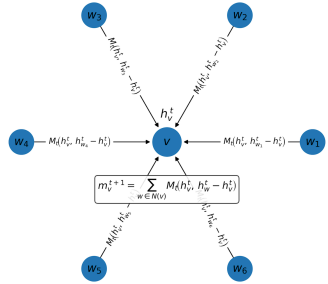
The graph has approximately Nk directed edges. Its topology is determined by learned features rather than only by the original coordinates.

Dynamic GNNs: EdgeConv

- Build a kNN graph in the current feature space h^t.
- The graph can change between layers.
- Construct the next representation from pairs of node features:

$$m_v^{t+1} = \sum_{w \in N_t(v)} M_t\left(h_v^t, h_w^t - h_v^t\right).$$

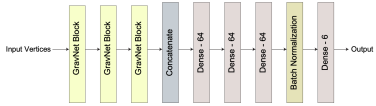
- Stack several such layers.
- Feature differences are useful when explicit edge attributes are absent, even on a static graph.

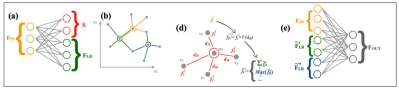


Yue Wang et al., “Dynamic graph CNN for learning on point clouds,” ACM Transactions on Graphics 38(5) (2019). Sum aggregation follows the formulation shown in the source lecture.

Dynamic GNNs: GravNet

- GravNet learns two spaces per layer:
  - a low-dimensional coordinate space S;
  - a higher-dimensional feature space F_{LR}.
- Build kNN in S and weight features by a negative exponential of distance.
- These weights provide a gradient to the coordinates, bringing useful neighbours together.
- Neighbour searches are faster in fewer dimensions.

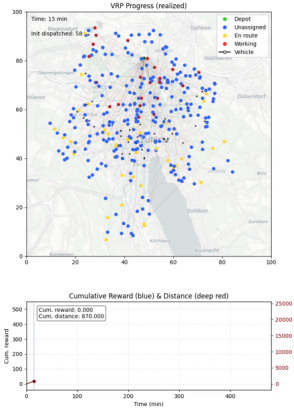




S. R. Qasim et al., “Learning representations of irregular particle-detector geometry with distance-weighted graph networks,” European Physical Journal C 79 (2019), 1–11.

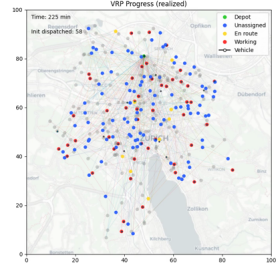
Applications of point clouds

- **Fluid dynamics**: points in space with learned interactions.
- **High-energy physics**: reconstruct interactions from particle clouds.
- **Social networks**: infer additional relationships.
- **Routing**: learn decisions over sets of locations.



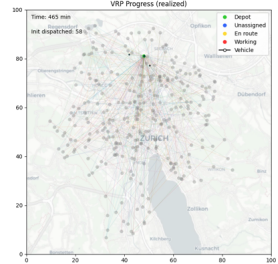
Applications of point clouds

- **Fluid dynamics**: points in space with learned interactions.
- **High-energy physics**: reconstruct interactions from particle clouds.
- **Social networks**: infer additional relationships.
- **Routing**: learn decisions over sets of locations.



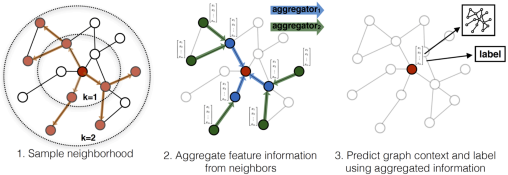
Applications of point clouds

- **Fluid dynamics**: points in space with learned interactions.
- **High-energy physics**: reconstruct interactions from particle clouds.
- **Social networks**: infer additional relationships.
- **Routing**: learn decisions over sets of locations.



GNNs at scale: GraphSAGE

- GraphSAGE: **sample and aggregate**.
- Sample a neighbourhood rather than processing every neighbour.
- Aggregate sampled features to build node representations.
- The authors also explored LSTM aggregation, which is not permutation invariant.
- Applications included citation and Reddit graphs.



1. Sample neighborhood
2. Aggregate feature information from neighbors
3. Predict graph context and label using aggregated information

Will Hamilton, Zhitao Ying and Jure Leskovec, "Inductive representation learning on large graphs," Advances in Neural Information Processing Systems 30 (2017).

Heterogeneous GNNs

- Many graphs contain multiple types of nodes and edges.
- A publication graph might contain:

Node types	Author, paper, venue
Edge types	Paper → cites → paper Author → writes → paper Paper → published in → venue

$$G = (V, E, T_V, T_E).$$

Summary

- GNNs provide a flexible way to learn from objects and their relationships.
- Message passing combines neighbourhood aggregation with learned feature updates.
- Graph-level, node-level and edge-level tasks use different prediction heads.
- Dynamic graphs extend these ideas to point clouds.
- GNNs remain useful across many domains alongside other architectures, including transformers.

Questions about a graph problem?

shahrukh.qasim@physik.uzh.ch

Part 2

Hands-on tutorial

Molecular classification with message passing

Find the Colab notebook

Open the notebook

`tinyurl.com/mr3vsmvt`

Open the full Google Colab link

- Make a copy before editing so that you can save your work.
- The notebook contains the training loop and a model scaffold.
- We will fill in the message-passing layer and the GNN.

The problem

- Molecules from an anti-HIV drug screen: atoms are nodes, bonds are edges.
- The GNN reads each whole molecule and predicts whether it is active against HIV.

Introduction to PyTorch

Graph-learning libraries

- PyTorch Geometric (PyG): `https://pyg.org/`
- TensorFlow GNN
- Deep Graph Library (DGL)

The supplied PyTorch example

- Start from the barebones CNN example.
- The training loop is already implemented.
- Define layers and their weights in the constructor.
- Apply those layers in `forward`.
- Familiar Keras layers have PyTorch counterparts.

Introduction to NetworkX

- NetworkX is a Python library for creating and analysing graphs and networks.
- It provides visualisation tools through Matplotlib.
- It includes classical graph algorithms, such as matching and clustering.
- We will use it to plot our graphs in the tutorial.

Implement the model

1. **Message-passing layer** First 15 minutes
   Implement messages, neighbourhood aggregation and node updates.
2. **Graph neural network** Next 15 minutes
   Stack message-passing layers, pool node features, and add dense layers for the classification head.
3. **Train and iterate**
   Run the existing training loop and refine the model.